CS5013 Final Project Report

BibMerge: one bibliography from many papers' references

S Naveen, CS26E010 Gowni Vamshi, CS26E005

Team Name:	Two Keys, One Paper
Repository:	`https://github.com/Sajjan-Naveen-87/BibMerge`
Submission commit:	`0777ef644a81b7b3d35760f9942397592b876534` (`final-submission`, 2026-11-06).
Docker image:	`https://drive.google.com/drive/folders/1ayGsAzr_j9qweRDTaFUQkwGJDQFHKqzP`
Demo video:	Same folder: recorded demo; the stakeholder's live demo will follow.
Tagline:	*Reads the `.bib` files or PDFs of several papers, finds references that name the same work, and writes one `.bib` in which every old citation key still works.*

AI-authorship clause. *We used AI tools to help draft parts of this report. The claims, numbers, commit references, stakeholder statements, and the Docker image are our own; the prose shaping may have been assisted. We can defend every sentence of this report, and every line of code in the submission commit, in the viva.*

1 Executive summary

Researchers who combine the references of several papers (for a survey, a thesis or a group library) find the same work under different citation keys, with titles, author lists and venues written differently. Our stakeholder, Prof. Ashrujit Ghoshal (CSE, IIT Madras), reconciles these by hand and has to redo the check every time a paper is added.

We shipped a Java 21 merge engine served as one JAR with a web app and a CLI. A user uploads several `.bib` files or papers' PDFs, answers "same paper / not the same" for the few borderline pairs, and downloads one `merged.bib`; old keys survive as biblatex `ids` aliases, so existing `\cite`s keep working, and a paper is added later by uploading the earlier `merged.bib` with it. 218 automated tests pass, and runs on real IACR ePrint and published IEEE papers gave no false merges. Scope change since the proposal: we added PDF input; storing decisions between sessions was dropped.

Stakeholder verdict: *pending*. We have asked Prof. Ghoshal for a final demo; a recorded demo is in the Drive folder, and the live one will be added.

2 Course concepts used in the project

Engine paths below are under `bibmerge-core/src/main/java/io/bibmerge/core/`.

- **Module 2 – Structured prompting.** Every significant prompt is logged verbatim beside its rewrite in our Context / Task / Constraints / Acceptance / Report template, with the commit it produced. File: `doc/prompts/README.md`. Commits: `2ae4d3b`, `eb0f911`.
- **Module 2 – Context injection.** Prompts cite design-doc sections (pipeline stages, ADRs, package layout) instead of restating them. File: `design/design.md`. Commit: `fc2ab05`.
- **Module 5 – Spec tests as executable contracts.** The partner's stages 7–9 were handed over as interfaces plus `@Disabled` tests stating exact behaviour; enabling them was the definition of done. File: `doc/tasks/vamshi-stages-7-9.md`, `cluster/UnionFind.java`. Commits: `2c49127` → `656cec2`.
- **Module 5 – Testing AI-written code.** Normalizers are checked against parameterised tables of real-world variants (16 DOI forms, 17 LaTeX accents); each proposal acceptance criterion maps to a named test. Files: `doc/VERIFICATION.md`, `normalize/DoiNormalizerTest.java` (test tree). Commit: `dad24a2`.
- **Module 4 – Debugging: reproduce, failing test, fix.** A real ePrint record carrying its published DOI bypassed a veto; the fix landed with its regression tests. File: `match/DecisionPolicy.java`. Commit: `be07587`.
- **Module 3 – Version control with AI.** Branch and PR per unit of work, CI (`mvn verify` + front-

end lint and build) on every PR, and a `Co-Authored-By` trailer on every AI-assisted commit. File: `.github/workflows/ci.yml`. Commits: `f1e5e6f`, `639c999` (PR integration).
- **Module 8 – Against unmaintainable AI output.** The Maven enforcer bans Spring from the engine module, so generated code cannot put merge logic in controllers. File: `bibmerge-core/pom.xml`. Commit: `f1e5e6f`.
- **Module 8 – Hybrid human–AI workflow.** Every pair decision records the rule that fired and its feature values; a reviewer's answer overrides the score and every veto. Files: `match/Constraints.java`, `cluster/ClusterResolver.java`, `bibmerge-web/src/App.tsx`. Commits: `7230470`, `2bb958b`.

3 AI-usage summary

Tools used. Claude Code (Claude Opus) for nearly all AI-assisted work: 35 of our 48 commits carry its co-author trailer. Gemini Pro only for the live modification at the mid-evaluation, where it was the one tool allowed. Roughly 95% / 5% (our estimate).

One substantive help. Claude Code drafted PDF input, `pdf/PdfReferenceExtractor.java` and `pdf/ReferenceParser.java`, with test tables for IEEE, Springer LNCS and ACM reference styles; it read 18 of 18 references from our compiled test PDFs as written and needed only small fixes for published two-column papers (`4711e78`, `8fbbfcd`).

One substantive failure caught in review. The AI-written key minter could mint a new key equal to another entry's retired key, so `\cite{oldkey}` would silently resolve to a different paper; each module passed its own tests, and our boundary bug hunt caught it before it reached `main` (`d6b9b9b`): *plausibly wrong* code silently corrupting persistent user data, in the terms of Module 8 (Limitations, Evaluation and Best Practices).

Net-effect sentence. Net help: the AI wrote most first drafts, and the tests and real-data runs we put around its output caught all six bugs in our debugging log.

4 Deployment and stakeholder handoff

Build and run.

```
docker load -i bibmerge-final.tar.gz          # file from the Drive link
docker run --rm -p 8091:8091 bibmerge:final   # web app: http://localhost:8091
docker run --rm bibmerge:final match demo/paper-a.bib demo/paper-b.bib  # CLI
```

Where the artefact lives after the course. The public GitHub repository (tag `final-submission`) and the Docker image on Drive. It is not yet on Prof. Ghoshal's machine; we will install `bibmerge.jar` there at the live demo if he wants to keep using it.

Stakeholder quote (one sentence, verbatim). Not yet available (demo pending); his words will go in the Drive folder.

5 Known limitations

- **Scale.** Blocking compares every pair of records: 2,000 records take 29 s and 1.5 GB, and the design target of 10,000 records in 10 s is not met.
- **No memory between sessions.** Reviewer answers last one browser session; adding a paper later works only if the user keeps and re-uploads the earlier `merged.bib`.
- **PDF input is approximate.** Scanned PDFs without a text layer give no references; on published IEEE papers 1 of 26 references came out with wrong fields, so fuzzy PDF matches go to review, never merge.

6 Individual contributions

- **S Naveen:** design document, scaffold and CI; engine stages 1–6 (parse, crossref, normalize, blocking, matching and decision rules); CLI, REST API and web UI; reviewer decisions and the review page; PDF input; edge-case hardening, `VERIFICATION.md` and the user guide. Stakeholder liaison.
- **Gowni Vamshi:** engine stages 7–9: `UnionFind` and `ClusterResolver` (clustering), `KeyMinter` and `GoldenRecordBuilder` (merged records), `BibWriter` (output), and their tests; reviewed and merged PRs #10–#12.